Implementación en Haskell de un *Sudoku Nonominós*
Programación Declarativa

Jorge Rodríguez de la Vega
C-412

. . . . . .

1. Descripción del problema

El *Sudoku Nonominós*, también conocido como el *sudoku geométrico*, es una variante de *Sudoku* en la que las regiones no son cuadradas sino que tienen formas *extrañas*, aunque todas están formadas por nueve celdas, de ahí el nombre: nonominós.

Por lo demás las reglas son iguales: números del 1 al 9, sin repetir en filas, columnas o regiones. Esta variación se aproxima al *Sudoku* regular, pero en lugar de nueve grupos de 3×3, tiene nueve grupos en formas irregulares de nonominó. Un nonominó es una figura de nueve espacios (*cuadraditos*) dispuestos en cualquier forma.

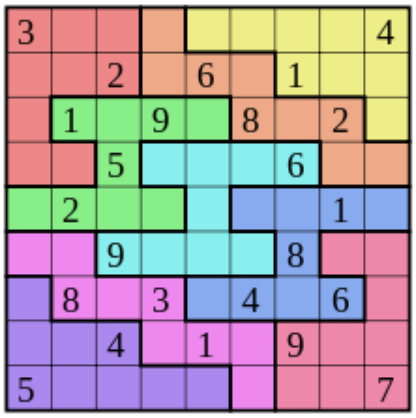


Figura 1: Ejemplo de un *Sudoku Nonominós*.

2. Modelación del problema

¿Cómo se definieron los *nonominós*, el *sudoku* y otros aspectos de la implementación?

Primeramente, se define la clase **Cell** de la siguinte forma:

```
1  data Cell = Cell {
2      row :: Int,
3      col :: Int,
```

```
        num :: Int,
        posInNonomino :: (Int, Int)
    } deriving (Show, Eq)

    instance Ord Cell where
        compare c1 c2 = compare (position2DTo1D c1) (position2DTo1D c2)

    position2DTo1D c = 9*(row c)+(col c)
```

- La idea es que cada **Cell** contenga los campos **row** y **col** inicializados en -1 ambos, y luego contenga un campo **posInNonomino** que representa la posición relativa de cada celda con respecto a la celda más arriba y más a la izquierda del nonominó.
- **row** y **col** irán cambiando de valores durante la ejecución del algoritmo: en todo momento representan la fila y la columna que la celda ocupa en el tablero respectivamente, pero si son -1 significa que esta celda aún no ha sido colocada en el tablero.
- **num** representa el número que posee la celda, puede ser 0 si está vacía o cualquier número del 1 al 9 si ya se le ha asignado un valor.
- Eventualmente, se necesitó ordenar las celdas según la posición que ocuparían en un tablero de *sudoku* leído de izquierda a derecha, y desde arriba hacia abajo, por lo que se definió el comparador usando **position2DTo1D**.

Auxiliándose de la clase **Cell**, se define la clase **Nonomino**:

```
data Nonomino = Nonomino {
    cells  :: [Cell],
    nonoID :: Int,
    fixed  :: Bool
    } deriving (Eq, Show)
```

- **cells** es una lista de **Cell**, la cual contiene las 9 celdas que se le definen inicialmente al nonominó. Cabe recordar que las celdas contienen su posición relativa a la celda más arriba y más a la izquierda de su nonominó.
- **nonoID** es un entero que representa a cada nonominó de manera única, y a cada uno se le asigna de manera automática el índice en el orden en el cual fueron definidos como **nonoID**.
- **fixed** es simplemente un valor de verdad para saber si el nonominó ya está fijado en el tablero a la hora de *ensamblar* el *sudoku*.

Finalmente, se define el último tipo de datos **Sudoku**:

```
data Sudoku = Sudoku {
    nonominoes :: [Nonomino]
} deriving (Eq)

instance Show Sudoku where
    show = showSudoku
```

- **nonominoes** representa la lista de *nonominós* de la entrada del problema.
- **showSudoku** es una función que se definió para obtener una representación legible del *sudoku*.

Todos los detalles más específicos de estas estructuras se encuentran en el código del fichero **DataStructures.hs**.

3. Un *sudoku* por *ensamblar* y *rellenar*

¿Qué ideas sería bueno seguir para resolver el problema?

Está claro que el problema a resolver está dividido en 2 subproblemas bien definidos: es necesario *acoplar* los *nonominós* de forma tal que formen un tablero válido de *sudoku*; y es necesario, una vez hecho esto, *rellenar* las casillas vacías de ese *sudoku*.

De aquí en adelante (tanto en texto como en código) estos problemas serán referidos como ***ensamblar*** y ***resolver*** el *sudoku*.

3.1. *Ensamblando* los *nonominós*

La idea para ensamblar los *nonominós* es bastante sencilla:

- Haciendo uso de la propiedad **fixed** de un **Nonomino**, es posible saber si está o no fijado.
- Como cada **Cell** conoce su posición relativa a la casilla más arriba y más a la izquierda (*primera*) del *Nonomino*, es sencillo calcular la *row* y *col* de cada celda si se conocen *row* y *col* de la *primera*. Si *a* y *b* denotan a **posInNonomino** de la casilla en cuestión y a (**row**, **col**) de la *primera* respectivamente:

```
1  calculateNewPosition a b =
2  let
3      newX = (fst a + fst b)
4      newY = (snd a + snd b)
5  in
6      (newX, newY)
```

- Tomando la posición de la *primera* casilla disponible en el tablero, es posible verificar si el primer *Nonomino* aún no fijado se puede colocar en ella (*i.e.* al calcular las posiciones de sus casillas, tomando la posición de la *primera* libre encontrada como la posición de la *primera* en el *Nonomino* estas están dentro del tablero y no *solapan* a ninguna otra casilla) y si es posible, colocarlo y probar con el siguiente; si no, volver al paso anterior y probar con otro. Esta idea se ve más clara al pensarla como un *backtrack*, y una posible implementación podría ser la siguiente:

```
1  assembleSudoku nns
2  | doneAssembling = Just Sudoku { nonominoes = nns }
3  | otherwise =
4      let nonos = [assembleSudoku(tryNonos) |
```

```
5               nnPos <- [0..8],
6               let (succeeded, tryNonos) = orderNonominoes nns nnPos,
7               succeeded,
8               validPartialSolution (tryNonos)]
9       in
10              firstNotNothing nonos
11  where
12      doneAssembling = (getTopMostLeftMostFreePosition nns) == (-1,-1)
```

- En cada paso se verifica no solo si el **Nonomino** *encaja* en una posición, sino también si al poner dicho **Nonomino** en dicha posición se genera un *sudoku* inválido.
- Si no es posible lograr esto en algún momento, entonces se devuelve [] y el *backtrack* se encarga de seguir probando otras formas de ensamblar los *nonominos*.
- En caso de que el sudoku no se pueda *ensamblar* con los *nonominós* de entrada, entonces se devuelve **Nothing**.
- **orderNonominoes** intenta ubicar un nonominó sin fijar en la *primera* posición vacía. Si lo logra, actualiza dicho *Nonomino* y las posiciones nuevas que toman sus casillas.
- **validPartialPosition** permite validar si una disposición de los nonominós podría o no generar un *sudoku* válido. Para ello, verifica que con dicha disposicion no quedan elementos repetidos por filas ni por columnas.
- **firstNotNothing** hace uso del tipo de datos **Maybe** de *Haskell*. De una lista de **Maybe** devuelve el primero que no sea **Nothing** (si no todos son **Nothing**, caso en el que devuelve **Nothing**).

Todos los detalles más específicos y la implementación de este subproblema se encuentran en el código del fichero **AssembleSudoku.hs**.

3.2. *Resolviendo el sudoku*

La idea seguida para *resolver* el *sudoku* se basa en la siguiente estrategia:

En cada paso, la casilla a la que se le debe fijar un valor para intentar resolver el *sudoku*, debe ser aquella casilla que menor cantidad de posibles números admita, y de estos valores que admite, escoger uno para computar el paso actual.

Seguir esta idea implica una reducción considerable en la cantidad de pasos que realiza el algoritmo, pues en cada paso se elige la manera *más corta* de generar las posibles soluciones. Otra gran ventaja que posee esta estrategia es que, como en cada paso se selecciona un número válido, no es necesario verificar en cada paso si el *sudoku* que se va *resolviendo* es válido o no.

Simplemente, si en algún paso no existen valore posibles para una casilla, se seleccionará la misma (pues se elige siempre la que menos valores posibles tenga) y si no está resuelto el *sudoku*, entonces se retorna y el *backtrack* se encargará de resolver el *sudoku* (de tener este solución).

Una implementación de esta idea podría ser la siguiente:

```
solveSudoku sud
    | isSolved sud = Just sud
    | otherwise =
        let
            bestChoice = smallestCell sud
            sol = [solveSudoku (changeCellValue sud bestChoice newValue) |
                newValue <- (getPossibleNums sud bestChoice)]
        in
            firstNotNothing sol
```

- **isSolved** devuelve **True** $\iff$ el *sudoku* ya está solucionado.
- **getPossibleNums** es una función que se encarga de computar los posibles valores que puede tomar una casilla vacía.
- **changeCellValue** se encarga de crear un *sudoku* igual al que se obtuvo en el paso anterior excepto por el nuevo valor que se le da a la casilla que se escogió para cambiar.
- **smallestCell** es una función que se encarga de escoger la casilla con menor cantidad de valores posibles a tomar.
- Análogo al subproblema anterior y con una finalidad semejante, aquí también se usa **firstNotNothing**.

Todos los detalles más específicos y la implementación de este subproblema se encuentran en el código del fichero **SolveSudoku.hs**.

4. Juntándolo todo...

Para comunicar ambos subproblemas y lograr la solución completa, se define una función **solution** en el fichero **Main.hs**:

```
solution
    | assembled == Nothing = error "¡Imposible resolver el Sudoku!"
    | otherwise =
        let
            Just solved = assembled
        in solveSudoku solved
    where assembled = assembleSudoku nns
```

nns es una lista de **Nonomino** que se define en el módulo **SudokuNonominoesDeclaration.hs**

Los detalles de cómo funciona la interacción con el proyecto están explicados detalladamente en el fichero **Readme.md** que se encuentra en la raíz del mismo.

5. Resolviendo el ejemplo inicial

A continuación se ofrece la solución del ejemplo inicial encontrada por las funciones analizadas anteriormente.

Primeramente, es necesario definir los *nonominós*; esto puede ser realizado de dos maneras: los *nonominós* pueden ser creados manualmente en el fichero **SudokuNonominoesDeclaration.hs**, o pueden ser generados el **Code generator for nonominoes.exe** siguiendo las instrucciones que aparecen en **Readme.md**. Para este ejemplo en particular, la definición de los *nonominós* quedaría así:

```
nono0:
300
002
0
00

nono1:
.1090
..5
0200
```

```
nono2:
.0006
..0
9000

nono3:
.0010
..8
0406
```

```
nono4:
00
.803
...010
.....0

nono5:
0
004
50000
```

```
nono6:
00004
..100
....0

nono7:
0
060
..802
....00
```

```
nono8:
.00
..0
900
007
```

3	5	8	1	9	6	2	7	4
4	9	2	5	6	7	1	3	8
6	1	3	9	7	8	4	2	5
1	7	5	8	4	2	6	9	3
8	2	6	4	5	3	7	1	9
2	4	9	7	3	1	8	5	6
9	8	7	3	2	4	5	6	1
7	3	4	6	1	5	9	8	2
5	6	1	2	8	9	3	4	7

Figura 2: Solución al ejemplo inicial.

```
*Main> saveSolution solution
*Main> solution
Just
 3:0  5:0  8:0  1:7  9:6  6:6  2:6  7:6  4:6
 4:0  9:0  2:0  5:7  6:7  7:7  1:6  3:6  8:6
 6:0  1:1  3:1  9:1  7:1  8:7  4:7  2:7  5:6
 1:0  7:0  5:1  8:2  4:2  2:2  6:2  9:7  3:7
 8:1  2:1  6:1  4:1  5:2  3:3  7:3  1:3  9:3
 2:4  4:4  9:2  7:2  3:2  1:2  8:3  5:8  6:8
 9:5  8:4  7:4  3:4  2:3  4:3  5:3  6:3  1:8
 7:5  3:5  4:5  6:4  1:4  5:4  9:8  8:8  2:8
 5:5  6:5  1:5  2:5  8:5  9:4  3:8  4:8  7:8
*Main>
```

Figura 3: Solución de este proyecto.

Si bien la figura 1 mostraba un ejemplo de *sudoku nonominó* sin resolver, la figura 2 muestra la solución del mismo, y la 3 muestra la solución que este proyecto le da a dicho ejemplo, con los *nonominós* delimitados en *rojo*.

La forma en la que se ejecutó el proyecto puede verse tanto en el **Readme.md** como en las líneas de la figura 3.